

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

1. Considerando um segundo algoritmo de ordenação, Quick Sort, revise o algoritmo e tente explicá-lo, usando diferentes perspectivas e representações:

O Quicksort é um algoritmo de ordenação que funciona de forma muito parecida com a organização de uma grande pilha de papéis por ordem alfabética ou numérica. A ideia central é “dividir para conquistar”.

1.1. Linguagem natural:

Instrução Principal: "Ordenar uma Lista"

A. O Ponto de Parada (Caso Base): Primeiro, olhe para a lista que você recebeu.

- Se a lista tiver nenhum item ou apenas um item, ela já está "ordenada".
- Nesse caso, devolva a lista como está. A tarefa terminou para esta lista.

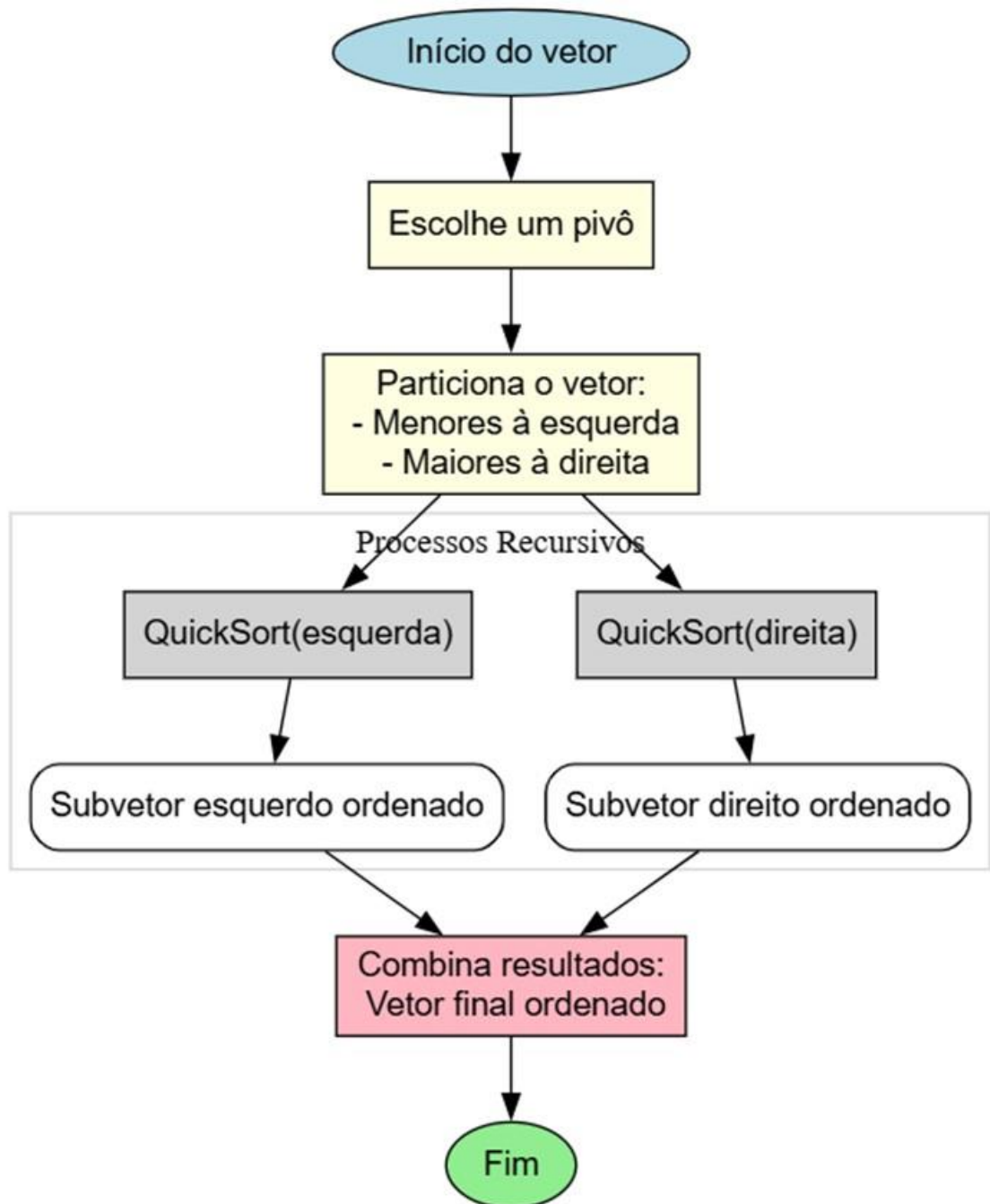
B. O Processo de Divisão (Se a lista tem 2+ itens): Se a lista tiver dois ou mais itens, faça o seguinte:

- 1. Escolha um "Pivô":
 - Pegue um item da lista para ser sua referência. Pode ser o primeiro, o último, ou um item aleatório e chama-o de "Pivô".
- 2. Crie duas Novas Listas Vazias:
 - Crie uma lista chamada "Menores".
 - Crie uma lista chamada "Maiores".
- 3. Compare e Distribua:
 - Agora, pegue todos os outros itens da lista (aqueles que não são o Pivô).
 - Para cada um desses itens, compare-o com o Pivô:
 - Se o item for menor ou igual ao Pivô, coloque-o na lista "Menores".
 - Se o item for maior que o Pivô, coloque-o na lista "Maiores".
- 4. Chame a Recursão:
 - Agora tem-se duas listas novas ("Menores" e "Maiores") que ainda estão bagunçadas.
 - Na lista "Menores", faça a recursão repetindo os itens A e B.
 - Na lista "Maiores", faça a recursão repetindo os itens A e B
 - Este procedimento seguirá até que o case base seja satisfeito.
- 5. Junte Tudo (Conquista):
 - Quando as instruções do passo (4) terminarem, terá a seguinte condição:
 1. A lista "Menores" (agora ordenada).
 2. O seu "Pivô" (que você guardou).
 3. A lista "Maiores" (agora ordenada).
 - Devolva a lista final juntando tudo nesta ordem: [Lista "Menores" ordenada] + [O "Pivô"] + [Lista "Maiores" ordenada]

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

Esta é a lista recebida está ordenada.

1.2. Diagramas livres



1.3. Pseudocódigo ou código:

Algoritmo "QuickSort"

// Função principal que executa o Quick Sort

Procedimento QuickSort(vetor[], inicio, fim)

Se (inicio < fim) Então

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

```
// Encontra a posição correta do pivô
pivoPosicao <- Particionar(vetor, inicio, fim)

// Ordena recursivamente as duas metades
QuickSort(vetor, inicio, pivoPosicao - 1)
QuickSort(vetor, pivoPosicao + 1, fim)

FimSe

FimProcedimento

// Função para realizar a partição do vetor
Função Particionar(vetor[], inicio, fim) : Inteiro
    pivo <- vetor[fim]          // Escolhe o último elemento como pivô
    i <- inicio - 1             // Índice do menor elemento

    Para j de inicio até fim - 1 Faça
        Se (vetor[j] <= pivo) Então
            i <- i + 1

            // Troca vetor[i] e vetor[j]
            aux <- vetor[i]
            vetor[i] <- vetor[j]
            vetor[j] <- aux

        FimSe

    FimPara

    // Coloca o pivô na posição correta
    aux <- vetor[i + 1]
    vetor[i + 1] <- vetor[fim]
    vetor[fim] <- aux

    Retorne i + 1

FimFunção


// Programa principal
Algoritmo Principal
    Declare vetor[10] : Inteiro
    Declare i : Inteiro

    // Leitura dos elementos do vetor
    Para i de 1 até 10 Faça
```

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

```
Escreva("Digite o elemento ", i, ": ")  
Leia(vetor[i])  
FimPara  
// Chamada do Quick Sort  
QuickSort(vetor, 1, 10)  
// Exibição do vetor ordenado  
Escreva("Vetor ordenado: ")  
Para i de 1 até 10 Faça  
    Escreva(vetor[i], " ")  
FimPara  
FimAlgoritmo
```

1.4. Diagrama formal

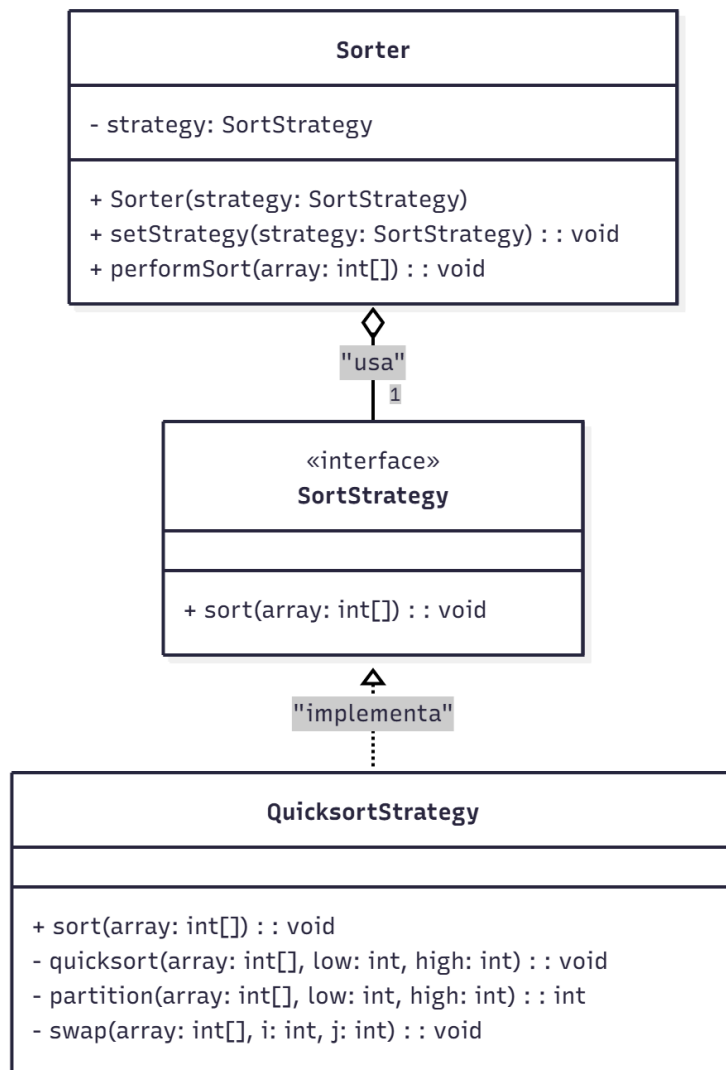


Figura 01: Diagrama de classe do algoritmo Quick Sort

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

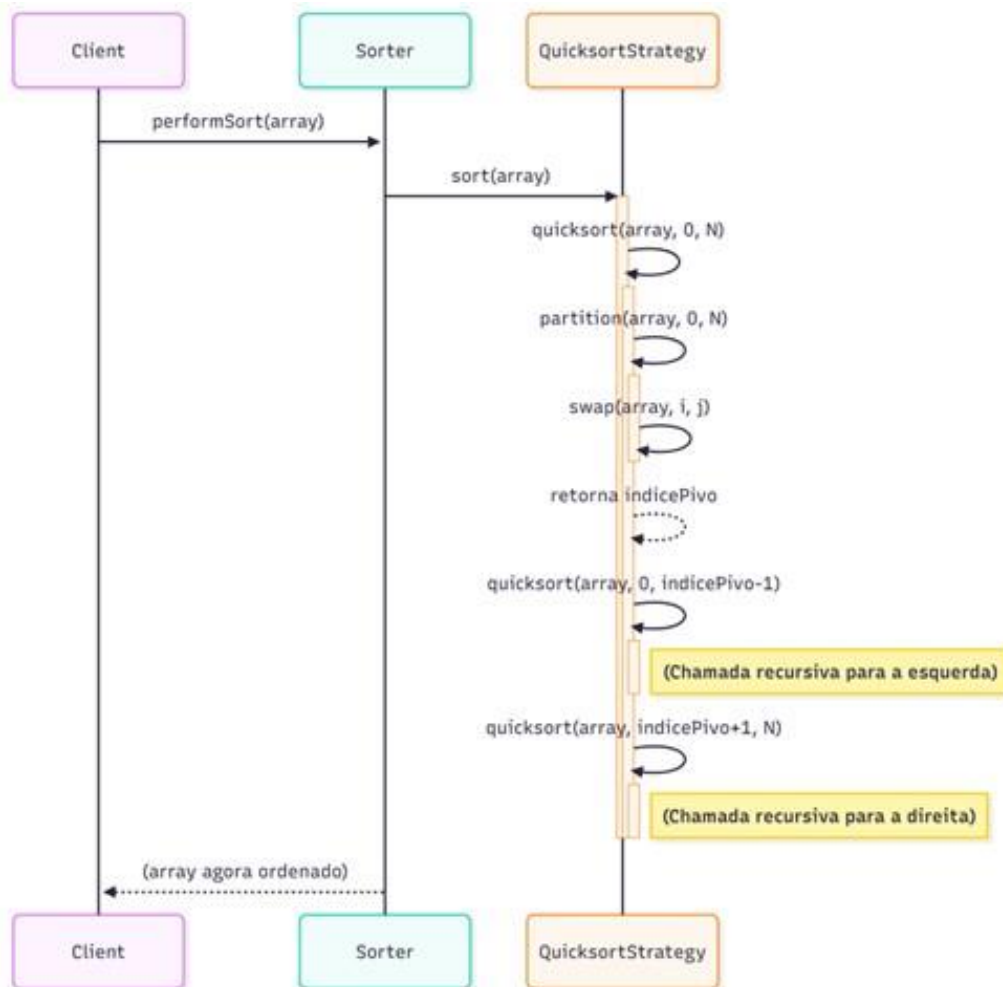


Figure 02: Diagrama de sequência do algoritmo Quick Sort

2. Compare as explicações dadas em cada perspectiva quanto a clareza e facilidade de entendimento, tipo de público-alvo, curva de aprendizado necessária para criar e compreender, recursos disponíveis para representação e contexto de aplicação (quando seria utilizada?).

A escolha da representação é uma decisão pragmática que impacta a clareza, a comunicação e a eficácia do *design* de software.

As fontes fornecem informações que permitem comparar as perspectivas de representação em relação aos critérios solicitados: **Linguagem Natural (e a Linguagem Ubíqua)**, **Representações Gráficas (Diagramas UML/Livres)** e **Código/Pseudo-código**.

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

Comparação de Perspectivas de Representação de Modelos Computacionais

Critério	Linguagem Natural (Linguagem Ubíqua)	Representações Gráficas (Diagramas UML/Livres)	Código/Pseudo-código
Clareza e Facilidade de Entendimento	Alta clareza, especialmente para a comunicação de significado e intenção. Requer Linguagem Ubíqua para precisão.	Alta clareza e entendimento instantâneo para a estrutura e relações. Mais fácil para visão global . Pode se tornar confuso com muitos elementos.	A expressão do comportamento (<i>como</i>) é clara, mas a intenção (<i>por que</i>) é mais difícil de ser transmitida. O significado conceitual (semântica) precisa ser interpretado.
Tipo de Público-Alvo	Especialistas de Domínio e toda a Equipe (incluindo desenvolvedores e <i>designers</i>).	Desenvolvedores (para modelos de <i>software</i>). Especialistas de Domínio (para modelos conceituais, se mantida a simplicidade).	Desenvolvedores/Técnicos . Especialistas de Domínio tendem a não ter ideia sobre bibliotecas, <i>frameworks</i> e persistência.
Curva de Aprendizado (Criar/Compreender)	Criar a Linguagem Ubíqua exige muito trabalho e foco. Compreender é fácil, desde que a linguagem seja consistente.	UML : Curva de aprendizado moderada. Diagramas simples são fáceis de entender, mas notações avançadas são complexas. Diagramas informais (<i>esboços</i>) exigem menos rigor.	A compreensão do código é inerente à fluência na linguagem de programação . <i>Designers</i> precisam de contato com o código para entender as restrições de implementação.
Recursos Disponíveis para Representação	Fala (uso implacável), Escrita (documentação concisa) e Diálogo .	UML (diagramas de classes, estados, atividades, etc.), Quadro Branco (<i>white-board</i>) para <i>esboços</i> rápidos, Ferramentas CASE (para <i>projetos</i> definitivos).	Linguagens de Programação OO (Java, C#, etc.), Pseudo-código , Testes de Unidade (para expressar <i>Asserções</i>).
Contexto de Aplicação (Quando Utilizada?)	Modelagem Contínua e Colaboração : Usada implacavelmente em todas as comunicações dentro de um Contexto Delimitado. Essencial para descobrir conceitos e resolver confusões.	Esboço : Usada para comunicação seletiva e dinâmica de ideias importantes. Projeto : Usada para especificação completa e definitiva (visando automação/redução da codificação manual). Documentação : Usada para fornecer um sumário gráfico e visão de alto nível.	Implementação : Usada para o resultado final e expressão literal do modelo. Verificação : Usada para entender o código legado ou obter <i>feedback</i> de <i>design</i> (TDD).

Detalhes Adicionais de Cada Perspectiva

2.1. Linguagem Natural e a Linguagem Ubíqua

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

A Linguagem Onipresente (Ubiquitous Language) é a linguagem natural estruturada em torno do modelo de domínio.

- Comunicação e Entendimento: Sua necessidade surge de uma barreira de comunicação fundamental entre desenvolvedores (que pensam em classes, métodos e padrões) e especialistas de domínio (que não têm ideia sobre *frameworks* ou bancos de dados). Um projeto enfrenta sérios problemas quando a equipe não compartilha uma linguagem comum.
- Significado: A linguagem ajuda a capturar o significado dos conceitos que os diagramas e o código não podem transmitir facilmente. No entanto, a linguagem humana é, por si só, terrivelmente imprecisa.
- Alinhamento: Requer que todos os membros da equipe usem a linguagem de forma consistente em toda comunicação e, crucialmente, no código. Uma mudança na linguagem deve ser reconhecida como uma mudança no modelo.

2.2. Representações Gráficas (Diagramas, incluindo UML)

Os diagramas são a sintaxe gráfica utilizada para expressar um modelo.

- Clareza e Abstração: Uma expressão gráfica de uma ideia é fácil de entender e ajuda todos a compartilharem instantaneamente a mesma visão sobre o tópico. Diagramas (como os da UML) são ótimas ferramentas para anotar os principais conceitos (como classes) e expressar as relações entre eles.
- Público e Curva: Diagramas de classes conceituais são muito úteis na exploração da linguagem de negócio, mas exigem que a notação seja mantida muito simples para que especialistas de domínio (não técnicos) possam acompanhá-la.
- Limitações: Diagramas grandes (com centenas de classes) são difíceis de ler e entender, mesmo pelos especialistas em *software*. Além disso, a UML não pode transmitir facilmente o significado dos conceitos ou o comportamento preciso das classes e as restrições, para o que se recorre a texto ou à OCL.

2.3. Código e Pseudo-código

O código é a manifestação final e mais detalhada do modelo.

- Clareza e Fidelidade: O código se torna uma expressão literal do modelo. Se o design não mapeia para o modelo de domínio, a exatidão do *software* é suspeita. Para tal, linguagens Orientadas a Objetos (OOP) são adequadas porque são baseadas no mesmo paradigma do modelo.
- Foco no Técnico: Linguagens procedurais oferecem suporte limitado para o *design* orientado a modelos, pois o significado dos dados e o comportamento só existe na mente do desenvolvedor, tornando o mapeamento domínio-código difícil de ser realizado.

ATIVIDADE: REPRESENTANDO MODELOS COMPUTACIONAIS

- Entendimento Tácito: Embora código bem escrito possa ser comunicativo, o nome do método (o *que* faz) é frequentemente mais claro do que o corpo do método (o *como* faz). O código, por si só, não expressa necessariamente a coisa certa.

Em resumo, para a Representação de Modelos Computacionais, o ideal pragmático é usar diagramas (especialmente como esboços) para comunicação rápida e visual da estrutura, o código para a especificação literal e testável da implementação, e a Linguagem Ubíqua (falada e escrita) como o veículo principal para garantir a precisão conceitual e a clareza para toda a equipe, incluindo o especialista de domínio. As representações textuais e gráficas atuam, assim, como assistentes úteis para as linguagens de programação textuais, que, no final, são o produto mais importante do projeto.